

関数

超サポ
愉快カンパニー

アシスト

関数とは？～処理に名前をつける～

同じ処理を何度も書かなくて済むようにまとめたものが関数です

関数とは？

同じ処理をまとめて
名前をつけたもの

何度も同じコードを
書かなくて済む！

```
def 関数名():  
    処理
```

実は今まで使ってきた！

今まで使ってきたものは
全部Pythonが用意した関数

```
print() → 表示する関数  
len()   → 長さを返す関数  
str()   → 変換する関数  
int()   → 変換する関数
```

自分でも作れる！

 関数は「処理に名前をつけたもの」。
Pythonが用意した関数だけでなく、自分でも作ることができます！

関数の定義 (def)・基本の形

def キーワードで関数を定義し、関数名 () で呼び出します

関数の定義：def 関数名(): で作る

```
def greet():  
    print("こんにちは！")
```

関数の呼び出し：関数名() で実行する

```
greet()    # → こんにちは！  
greet()    # → こんにちは！ (何度でも使える)  
greet()    # → こんにちは！
```

書き方のルール

def のあとに関数名を書く
関数名のあとに () をつける
コロンの (:) を忘れずに
処理はインデント (4マス下げ)

() の意味

() がない → 何も起きない
() がある → 関数が実行される

print と print() の
違いと同じ考え方！

変数との違い

変数はデータを、関数は処理をまとめるものです

変数

値（データ）を入れる箱

```
name = "山田"  
score = 100
```

- データを保存しておく
- 呼び出すと値が返る

イメージ：箱・引き出し

関数

処理（命令）をまとめたもの

```
def greet():  
    print("こんにちは！")
```

- 処理を保存しておく
- 呼び出すと処理が実行される

イメージ：レシピ・機械



変数 = データを入れる箱 関数 = 処理をまとめたもの。
どちらも「名前をつけて後で使う」点は同じ！

引数～関数に値を渡す～

引数を使うと、渡した値に応じて処理を変えることができます

```
# 引数なし：誰に対しても同じ挨拶しかできない
```

```
def greet():
```

```
    print("こんにちは！")
```

```
# 引数あり：渡した値に応じて処理が変わる
```

```
def greet(name):    # name が引数
```

```
    print(f"{name}さん、こんにちは！")
```

```
greet("山田")    # → 山田さん、こんにちは！
```

```
greet("佐藤")    # → 佐藤さん、こんにちは！
```

引数とは

関数を呼び出すときに
渡す値のこと

関数の () の中に書く
複数渡すこともできる
def add(a, b):

引数を使うメリット

同じ関数でも
渡す値によって
結果が変わる

→ 1つの関数で
様々な場面に対応できる

戻り値 (return) ～結果を返す～

returnを使うと関数の処理結果を呼び出し元に返すことができます

```
# returnなし：画面に表示するだけ
def greet(name):
    print(f"{name}さん、こんにちは！")
```

```
# returnあり：結果を次の処理に渡せる
def add(a, b):
    return a + b    # 結果を返す
```

```
result = add(10, 20)    # → 30 が返ってくる
print(result)           # → 30
print(result * 2)       # → 60 (さらに計算できる！)
```

returnなし

画面に表示するだけ
結果を他の処理に使えない

returnあり

結果を変数に入れられる
さらに計算・加工できる

関数を使うメリット～再利用性～

関数を使うと修正が1か所で済み、コードが読みやすくなります

✗ 関数なし

```
# 税込価格を3回計算  
print(1000 * 1.1)  
print(2000 * 1.1)  
print(3000 * 1.1)
```

消費税が10%→8%に変わったら
3か所全部直す必要がある😱

100か所あったら...？

✓ 関数あり

```
def tax_price(price):  
    return price * 1.1
```

```
print(tax_price(1000))  
print(tax_price(2000))  
print(tax_price(3000))
```

1.1 を 1.08 に変えるだけ！
修正は1か所だけ😊

💡 関数のメリット：①同じ処理を何度も書かなくていい
②修正が1か所で済む ③処理に名前がつくので読みやすい

やってみよう:関数

以下のコードを書いて実行してみましょう

```
# ① 自分の名前を挨拶する関数を作ってみよう
```

```
def greet(name):  
    print(f"{name}さん、こんにちは！")
```

```
greet("〇〇")    # 自分の名前を入れてみよう
```

```
# ② 2つの数を足し算する関数を作ってみよう
```

```
def add(a, b):  
    return a + b
```

```
result = add(10, 20)  
print(result)    # → 30
```

```
# ③ 税込価格を計算する関数を作ってみよう
```

```
def tax_price(price):  
    return price * 1.1
```

```
print(tax_price(1000))    # → 1100.0
```


まとめ

関数とは

同じ処理をまとめて名前をつけたもの。def で定義し、関数名() で呼び出す

引数

関数を呼び出すときに渡す値。渡した値に応じて処理を変えられる

戻り値 (return)

関数の処理結果を返す。returnがあると結果を変数に入れたりさらに計算できる

メリット

①同じ処理を何度も書かなくていい ②修正が1か所で済む ③コードが読みやすくなる